



青岛科技大学
Qingdao University of Science & Technology

数据结构与算法分析2

第7章 动态规划2—算法案例

梁宏涛

lht@qust.edu.cn

周二&周四上午

1

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

回顾：

动态规划其实就是，给定一个问题，我们把它拆成一个个子问题，直到子问题可以直接解决。然后呢，把子问题答案保存起来，以减少重复计算。再根据子问题答案反推，得出原问题解的一种方法。

一般这些子问题很相似，可以通过函数关系式（DP方程）递推出来。动态规划就致力于解决每个子问题一次，减少重复计算。其核心思想就是：拆分子问题，记住过往，减少重复计算

动态规划，DP数组记录子问题的结果，减少子问题重复求解问题。

如果一个问题，可以把所有可能的答案穷举出来，并且穷举出来后，发现存在重叠子问题，就可以考虑使用动态规划。如最长递增子序列、最小编辑距离、背包问题、凑零钱问题等等，都是动态规划的经典应用场景

<https://blog.csdn.net/Gherbirthday0916/article/details/127974834>

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

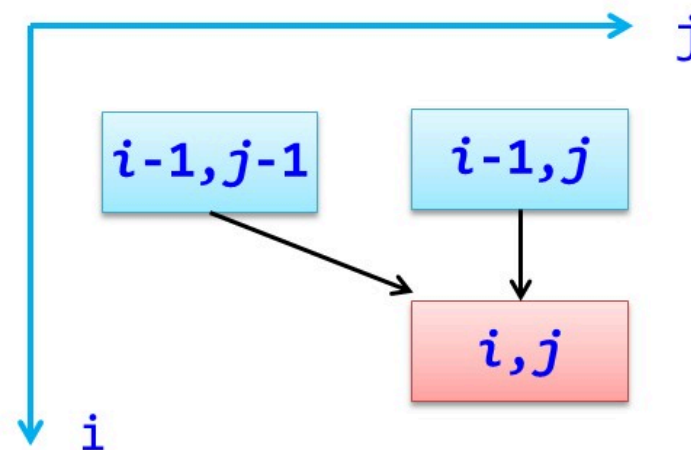
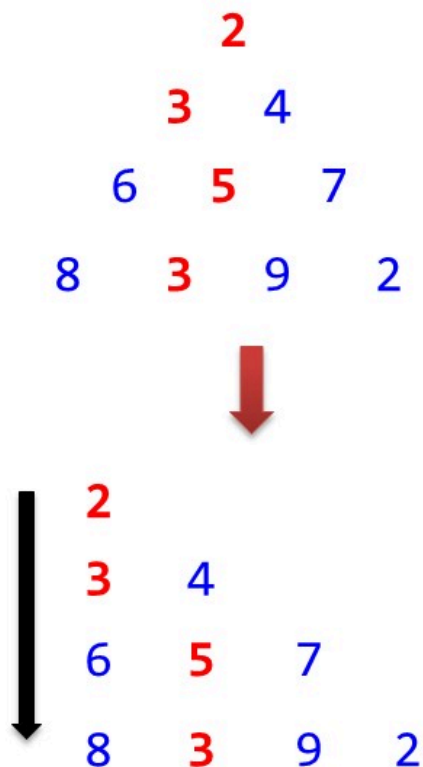
7.4 三角形最小路径

- 给定一个高度为 n 的整数三角形，求从顶部到底部的最小路径和及其一条最小路径，从每个整数出发只能向下移动到相邻的整数。
- 例如，一个 $n=4$ 的三角形，输出的最小路径和是13，一条最小路径是2, 3, 5, 3。



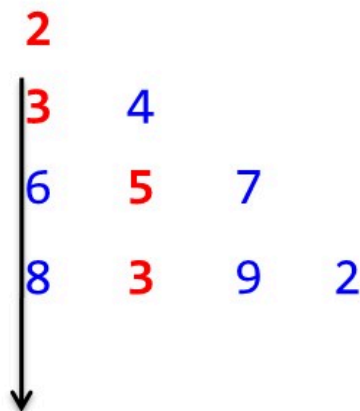
解

解法1：问题求解—自顶向下

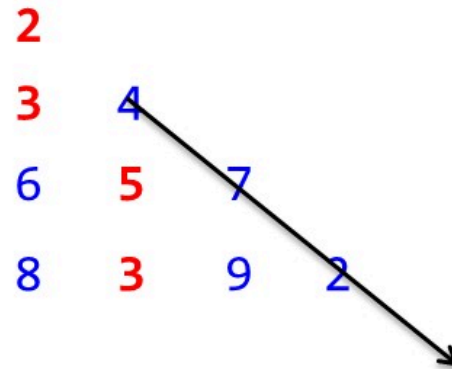


7

- 设计二维动态规划数组 dp ， $dp[i][j]$ 表示从顶部 $a[0][0]$ 到达 (i, j) 位置的最小路径和。
- 起始位置只有 $(0, 0)$ ，初始化为 $dp[0][0]=a[0][0]$ 。
- 这里有如下两个边界：
 - ① 对于 $j=0$ 即第0列的任意位置 $(i, 0)$ ，只有垂直方向到达的一条路径，此时有 $dp[i][0]=dp[i-1][0]+a[i][0]$ 。

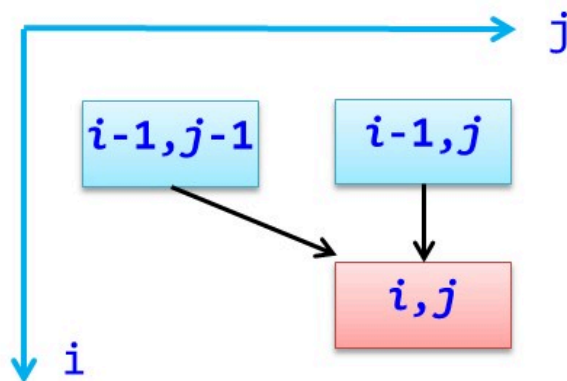


② 对于 $i=j$ 即对角线上的任意位置 (i, i) ，只有左斜方向到达的一条路径，此时有 $dp[i][i] = dp[i-1][i-1] + a[i][i]$ 。



■ 其他两条到达 (i, j) 位置的路径，最小路径和

$$dp[i][j] = \min(dp[i-1][j-1], dp[i-1][j]) + a[i][j]。$$



状态转移方程



$dp[0][0]=a[0][0]$

初始条件

$dp[i][0]=dp[i-1][0]+a[i][0]$

第0列的边界情况, $1 \leq i \leq n-1$

$dp[i][i]=dp[i-1][i-1]+a[i][i]$

对角线的边界情况, $1 \leq i \leq n-1$

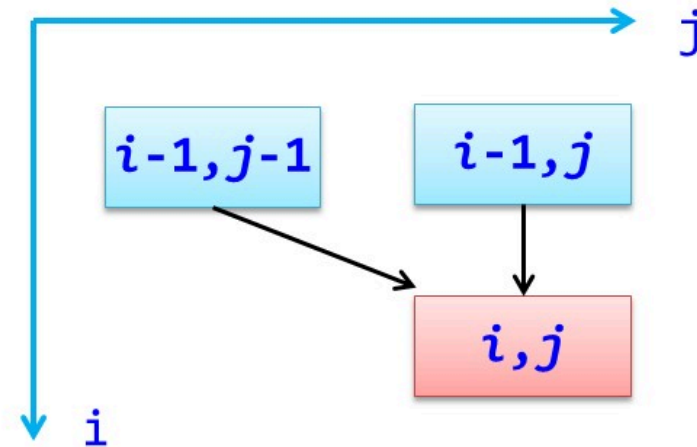
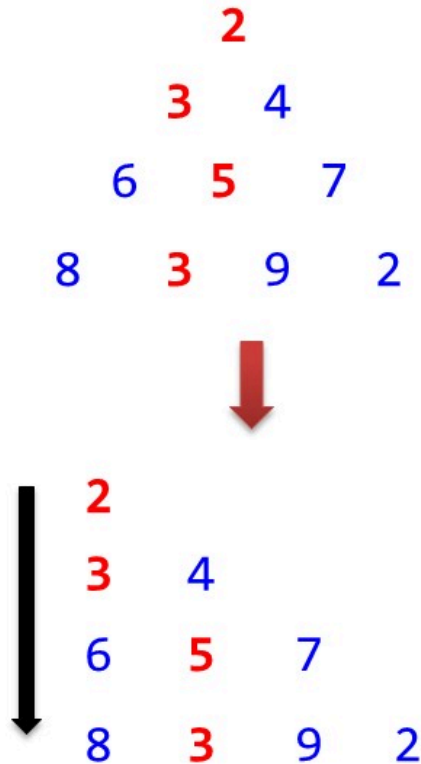
$dp[i][j]=\min(dp[i-1][j-1], dp[i-1][j])+a[i][j]$

$i > 1$ 的其他有两条达到路径

在dp数组的第 $n-1$ 行中求出的最小元素 $ans=dp[n-1][minj]$ 。

主观题 10分

根据以上边界递推式，求解DP，拍照上传。




```

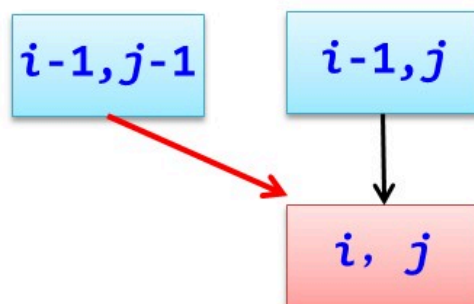
int minpathsum(vector<vector<int>>&a){ //自顶向下求最小路径和
    int n=a.size();
    int dp[n][n]; //二维动态规划数组
    dp[0][0]=a[0][0];
    for(int i=1;i<n;i++) //考虑第0列的边界
        dp[i][0]=dp[i-1][0]+a[i][0];
    for (int i=1;i<n;i++) //考虑对角线的边界
        dp[i][i]=a[i][i]+dp[i-1][i-1];
    for(int i=2;i<n;i++) { //考虑其他结点
        for(int j=1;j<i;j++)
            dp[i][j]=min(dp[i-1][j-1],dp[i-1][j])+a[i][j];
    }

    int ans=dp[n-1][0];
    for (int j=1;j<n;j++) //求出最小ans
        ans=min(ans,dp[n-1][j]);
    return ans;
}

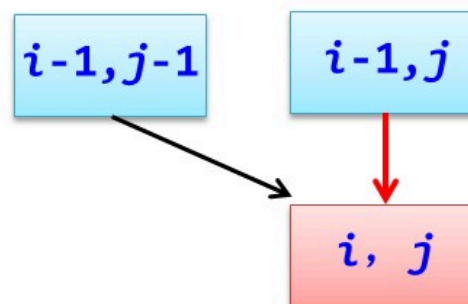
```

dp $\Rightarrow$ path

- 那么如何找到一条最小和的路径呢？
- 设计一个二维数组 **pre**, **pre[i][j]** 表示到达 **(i, j)** 位置时最小路径上的前驱位置, 用 **pre[i][j]** 记录前驱位置的列号即可。



pre[i][j]=j-1



pre[i][j]=j

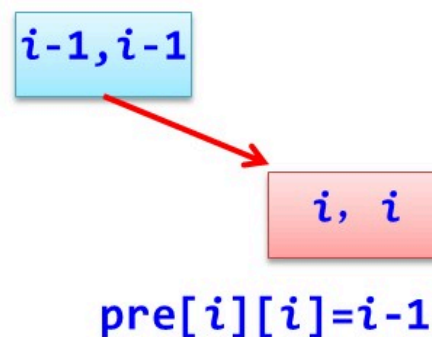
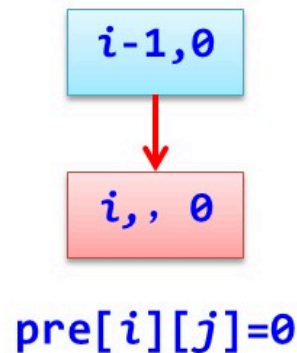
- 在求出 **ans** 后, 通过 **pre[n-1][minj]** 推导求出反向路径 **path**, 逆向输出得到一条最小和的路径。

```

void minpathsum1(vector<vector<int>>&a) {
    int n=a.size();
    int dp[n][n];
    int pre[n][n];
    dp[0][0]=a[0][0];
    for(int i=1;i<n;i++){
        dp[i][0]=dp[i-1][0]+a[i][0];
        pre[i][0]=0;
    }
    for (int i=1;i<n;i++){
        dp[i][i]=a[i][i]+dp[i-1][i-1];
        pre[i][i]=i-1;
    }
}

```

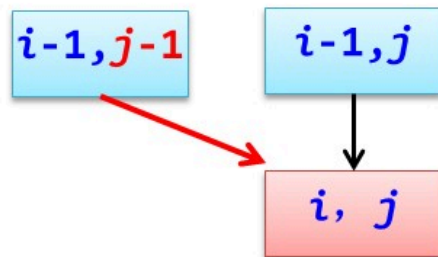
//二维动态规划数组
 //二维路径数组
 //考虑第0列的边界
 //考虑对角线的边界



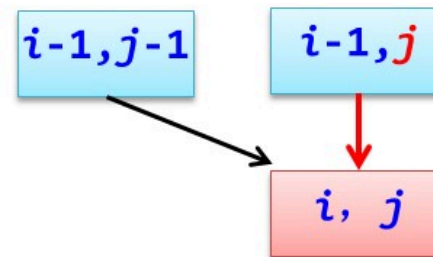
```

for(int i=2;i<n;i++) {           //考虑其他有两条达到路径的结点
    for(int j=1;j<i;j++) {
        if(dp[i-1][j-1]<dp[i-1][j]) {
            pre[i][j]=j-1;
            dp[i][j]=a[i][j]+dp[i-1][j-1];
        }
        else {
            pre[i][j]=j;
            dp[i][j]=a[i][j]+dp[i-1][j];
        }
    }
}

```



$pre[i][j]=j-1$



$pre[i][j]=j$


```

int ans=dp[n-1][0];
int minj=0;
for (int j=1;j<n;j++) {           //求最小ans和对应的列号minj
    if (ans>dp[n-1][j]) {
        ans=dp[n-1][j];
        minj=j;
    }
}

printf("求解结果\n");
printf("  最小路径和=%d\n",ans);
int i=n-1;
vector<int> path;                  //存放一条反向路径
while (i>=0) {                    //从(n-1,minj)位置反推
    path.push_back(a[i][minj]);
    minj=pre[i][minj];             //最小路径在前一行中的列号
    i--;                           //在前一行查找
}

```

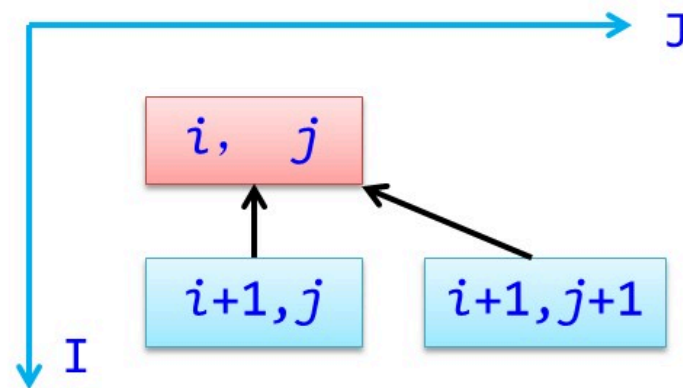
16


```
reverse(path.begin(),path.end());    //逆置path
printf("  一条最小路径: ");
for(int i=0;i<path.size();i++)
    printf(" %d",path[i]);
printf("\n");
}
```

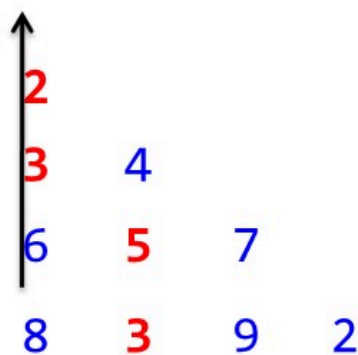
解法2：问题求解—自底向上

↑

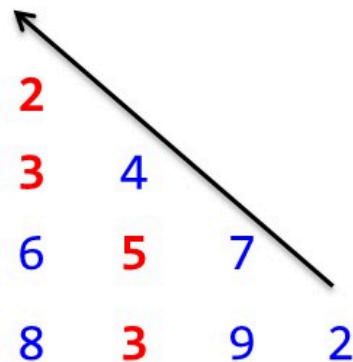
2			
3	4		
6	5	7	
8	3	9	2



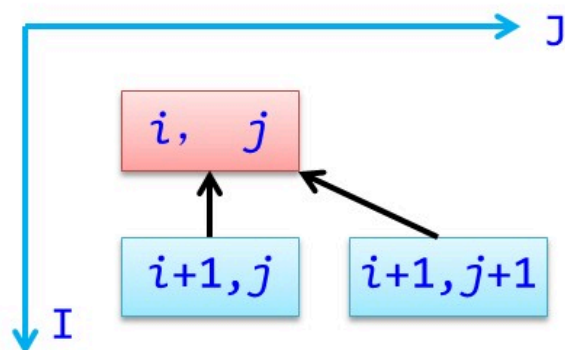
- 设计二维动态规划数组 dp ，其中 $dp[i][j]$ 表示从底部到达 (i, j) 位置的最小路径和。
- 起始位置 $(n-1, *)$ ，初始化为 $dp[n-1][j]=a[n-1][j]$ 。
- 同样有如下两个边界：
 - ① 对于 $j=0$ 即第0列的任意位置 $(i, 0)$ ，只有垂直方向到达的一条路径，此时有 $dp[i][0]=dp[i+1][0]+a[i][0]$ 。



① 对于 $i=j$ 即对角线上的任意位置 (i, i) ，只有左斜方向到达的一条路径，此时有 $dp[i][i] = dp[i+1][i+1] + a[i][i]$ 。



- 其他两条到达 (i, j) 位置的路径，最小路径和
 $dp[i][j] = \min(dp[i+1][j+1], dp[i+1][j]) + a[i][j]$ 。



状态转移方程



$$dp[n-1][j]=a[n-1][j]$$

初始条件

$$dp[i][0]=dp[i+1][0]+a[i][0]$$

第0列的边界情况, $0 \leq i \leq n-2$

$$dp[i][i]=dp[i+1][i+1]+a[i][i]$$

对角线的边界情况, $0 \leq i \leq n-2$

$$dp[i][j]=\min(dp[i+1][j], dp[i+1][j+1])+a[i][j]$$

$i < n-1$ 两条达到路径

由于第0行只有一个元素, $dp[0][0]$ 就是最终的最小路径和。

主观题 10分

根据自底向上边界递推式，求解DP，拍照上传。

$$dp[n-1][j]=a[n-1][j]$$

初始条件

$$dp[i][0]=dp[i+1][0]+a[i][0]$$

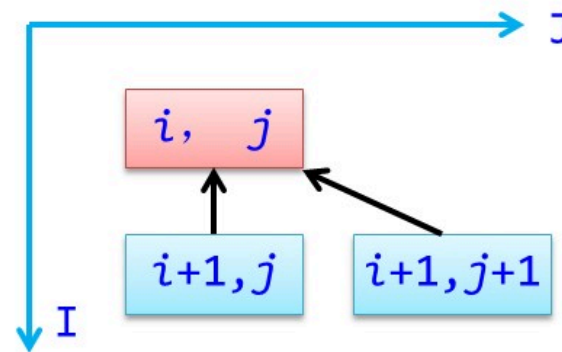
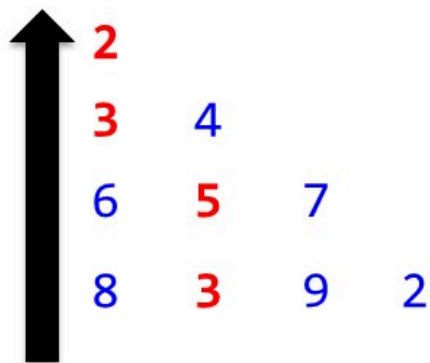
第0列的边界情况, $0 \leq i \leq n-2$

$$dp[i][i]=dp[i+1][i+1]+a[i][i]$$

对角线的边界情况, $0 \leq i \leq n-2$

$$dp[i][j]=\min(dp[i+1][j], dp[i+1][j+1])+a[i][j]$$

$i < n-1$ 两条达到路径



`dp[n-1][j]=a[n-1][j]`

初始条件

`dp[i][0]=dp[i+1][0]+a[i][0]`

第0列的边界情况, $0 \leq i \leq n-2$

`dp[i][i]=dp[i+1][i+1]+a[i][i]`

对角线的边界情况, $0 \leq i \leq n-2$

`dp[i][j]=min(dp[i+1][j], dp[i+1][j+1])+a[i][j]` $i < n-1$ 两条达到路径

```
int minpathsum2(vector<vector<int>>&a){
    int n=a.size();
    int dp[n][n];
    for(int j=0;j<n;j++)
        dp[n-1][j]=a[n-1][j];
    for(int i=n-2;i>=0;i--)
        dp[i][0]=dp[i+1][0]+a[i][0];
    for (int i=n-2;i>=0;i--)
        dp[i][i]=a[i][i]+dp[i+1][i+1];
    for(int i=n-2;i>=0;i--){
        for(int j=0;j<a[i].size();j++)
            dp[i][j]=min(dp[i+1][j+1],dp[i+1][j])+a[i][j];
    }
    return dp[0][0];
}
```

23

自底向上算法空间优化

在自底向上算法中阶段*i*（指求第*i*行的dp）仅仅与阶段*i+1*相关，采用降维滚动数组方式，将dp由二维数组改为一维数组。

```
int minpathsum3(vector<vector<int>>&a) {  
    int n=a.size();  
    int dp[n];                      //一维动态规划数组  
    memset(dp,0,sizeof(dp));  
    for(int i=n-1;i>=0;i--) {  
        for(int j=0;j<a[i].size();j++) {  
            if(j<a[i].size()-1)  
                dp[j]=min(dp[j],dp[j+1])+a[i][j];  
            else  
                dp[j]+=a[i][j];  
        }  
    }  
    return dp[0];  
}
```


提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

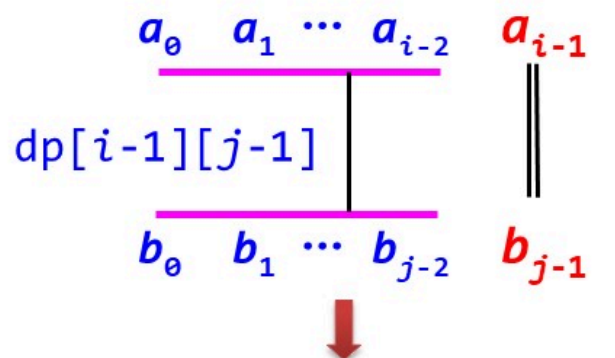
7.13矩阵连乘问题

7.5 最长公共子序列

- 一个字符串的子序列是指从该字符串中随意地（不一定连续）去掉若干个字符（可能一个也不去掉）后得到的字符序列。
- 例如"ace"是"abcde"的子序列，但"aec"不是"abcde"的子序列。
- 给定两个字符串 a 和 b ，称字符串 c 是 a 和 b 的公共子序列，是指 c 同是 a 和 b 的子序列。该问题是求两个字符串 a 和 b 的最长公共子序列（LCS）。

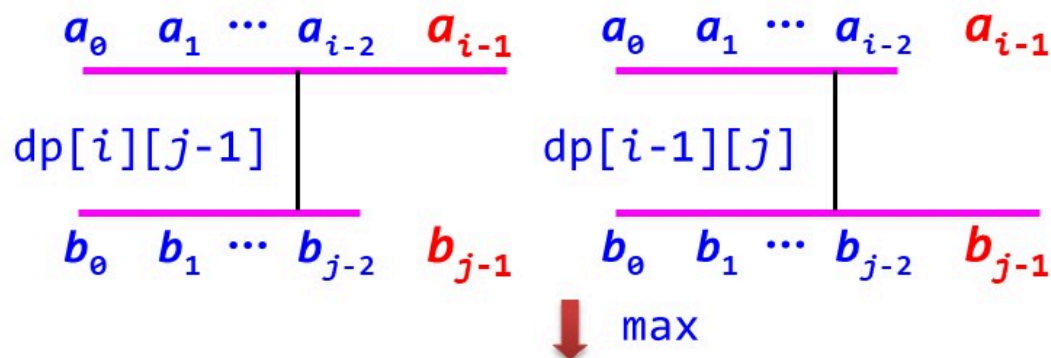
解

设计二维动态规划数组 dp ，其中 $dp[i][j]$ 为 $(a_0, a_1, \dots, a_{i-1})$ 和 $(b_0, b_1, \dots, b_{j-1})$ 的最长公共子序列长度，求 $dp[i][j]$ 的两种情况。



$$dp[i][j] = dp[i-1][j-1] + 1$$

$$(a) \quad a_{i-1} = b_{j-1}$$



$$dp[i][j] = \max(dp[i][j-1], dp[i-1][j])$$

$$(b) \quad a_{i-1} \neq b_{j-1}$$

状态转移方程

$dp[0][0]=0$

初始条件

$dp[i][0]=0$

边界情况

$dp[0][j]=0$

边界情况

$dp[i][j]=dp[i-1][j-1]+1$

$a[i-1]=b[j-1]$

$dp[i][j]=\max\{dp[i][j-1], dp[i-1][j]\}$

$a[i-1]\neq b[j-1]$

在求出 dp 数组后， $dp[m][n]$ 就是 a 和 b 的最长公共子序列长度。

主观题 10分

$A=\{1,2,5,7,0\}$, $b=\{1,5,7,3,4\}$,求解DP, 拍照上传。

$dp[0][0]=0$

初始条件

$dp[i][0]=0$

边界情况

$dp[0][j]=0$

边界情况

$dp[i][j]=dp[i-1][j-1]+1$

$a[i-1]=b[j-1]$

$dp[i][j]=\max\{dp[i][j-1], dp[i-1][j]\}$

$a[i-1]\neq b[j-1]$

在求出dp数组后, $dp[m][n]$ 就是a和b的最长公共子序列长度。

```

vector<vector<int>> dp;           //二维动态规划数组
int LCSlength(string&a,string&b) { //求dp和LCS长度
    int m=a.size();             //m为a的长度
    int n=b.size();             //n为b的长度
    dp=vector<vector<int>>(m+1,vector<int>(n+1,0));
    dp[0][0]=0;
    for (int i=0;i<=m;i++)       //将dp[i][0]置为0,边界条件
        dp[i][0]=0;
    for (int j=0;j<=n;j++)       //将dp[0][j]置为0,边界条件
        dp[0][j]=0;
}

```



```
for (int i=1;i<=m;i++) {  
    for (int j=1;j<=n;j++) { //两重for循环处理a、b的所有字符  
        if (a[i-1]==b[j-1]) //情况(1)  
            dp[i][j]=dp[i-1][j-1]+1;  
        else //情况(2)  
            dp[i][j]=max(dp[i][j-1],dp[i-1][j]);  
    }  
}  
return dp[m][n];  
}
```

当求出 dp 后如何利用 dp 求一个最长公共子序列呢？分析状态转移方程最后两行的计算过程可以看出：

- ① 若 $dp[i][j]=dp[i-1][j-1]+1$ ，则有 $a[i-1]=b[j-1]$ ，也就是说 $a[i-1]/b[j-1]$ 是LCS中的字符。
- ② 若 $dp[i][j]=dp[i][j-1]$ ，则有 $a[i-1]\neq b[j-1]$ ，也就是说 $a[i-1]/b[j-1]$ 不是LCS中的字符。
- ③ 若 $dp[i][j]=dp[i-1][j]$ ，则有 $a[i-1]\neq b[j-1]$ ，同样 $a[i-1]/b[j-1]$ 不是LCS中的字符。

$$dp[i][j]=dp[i-1][j-1]+1$$

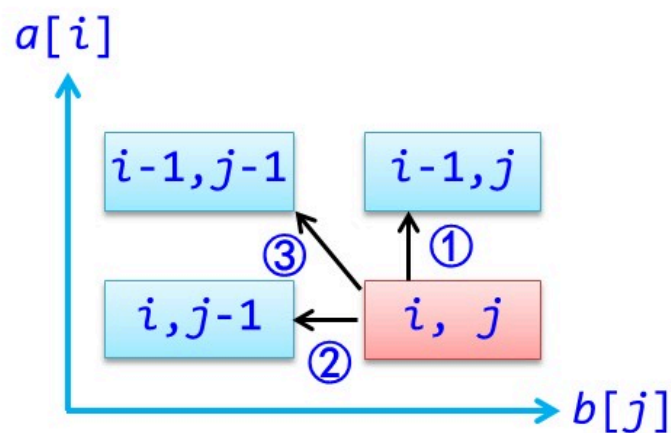
$$a[i-1]=b[j-1]$$

$$dp[i][j]=\max\{dp[i][j-1], dp[i-1][j]\}$$

$$a[i-1]\neq b[j-1]$$

用 **string** 容器 **subs** 存放一个 **LCS**, $i=m$, $j=n$ 开始向 **subs** 中添加 $k=dp[m][n]$ 个字符, 归纳为如下 3 种情况:

- ① 若 $dp[i][j]=dp[i-1][j]$ (即当前 **dp** 元素值等于上方相邻元素值), 移到上一行即 i 减 1, 此时 $a[i-1]/b[j-1]$ 不是 **LCS** 字符。
- ② 若 $dp[i][j]=dp[i][j-1]$ (即当前 **dp** 元素值等于左边相邻元素值), 移到左一列即 j 减 1, 此时 $a[i-1]/b[j-1]$ 不是 **LCS** 字符。
- ③ 其他情况只能是 $dp[i][j]=dp[i-1][j-1]+1$, 移到左上方即 i 减 1 同时 j 减 1, 此时 $a[i-1]/b[j-1]$ 是 **LCS** 的字符, 将 $a[i-1]/b[j-1]$ 添加到 **subs** 中。



例如, $a="abcbdb"$, $m=6$, $b="acbbabdbb"$, $n=9$ 。

$\begin{matrix} b \\ a \end{matrix}$		a	c	b	b	a	b	d	b	b	
		0	1	2	3	4	5	6	7	8	9
a	0	0	0	0	0	0	0	0	0	0	0
b	1	0	1	1	1	1	1	1	1	1	1
c	2	0	1	1	2	2	2	2	2	2	2
b	3	0	1	2	2	2	2	2	2	2	2
d	4	0	1	2	3	3	3	3	3	3	3
b	5	0	1	2	3	3	3	3	4	4	4
	6	0	1	2	3	4	4	4	4	5	5

- ① 若 $dp[i][j]=dp[i-1][j]$, 移到上一行, $a[i-1]/b[j-1]$ 不是 LCS 字符。
- ② 若 $dp[i][j]=dp[i][j-1]$, 移到左一列, $a[i-1]/b[j-1]$ 不是 LCS 字符。
- ③ 若 $dp[i][j]=dp[i-1][j-1]+1$, 移到左上方, $a[i-1]/b[j-1]$ 是 LCS 的字符。

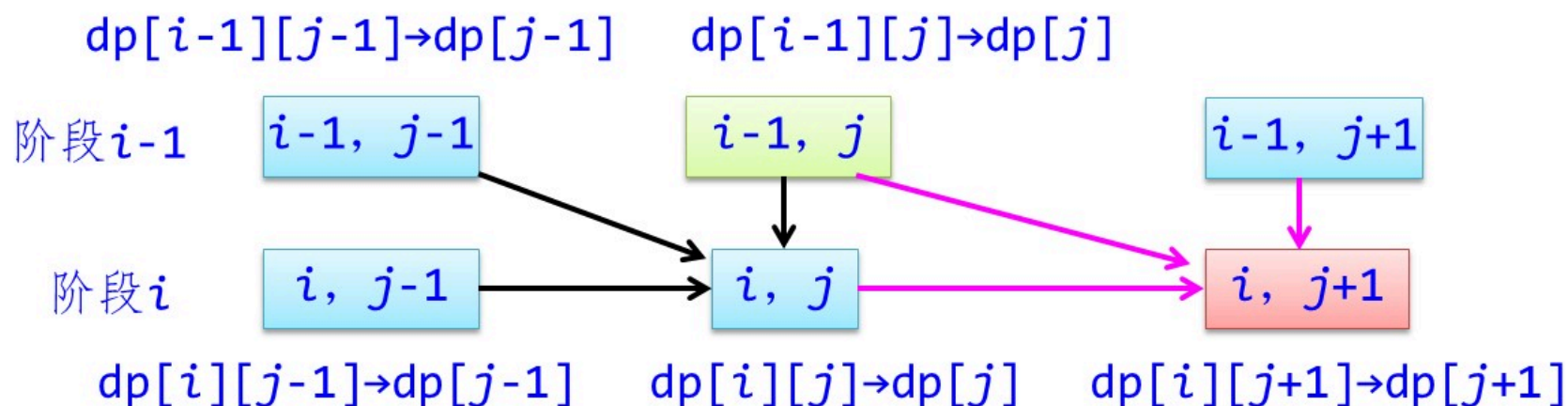

```

string getasubs(string&a,string&b) {
    string subs="";           //存放一个LCS
    int m=a.size();           //m为a的长度
    int n=b.size();           //n为b的长度
    int k=dp[m][n];           //k为最长公共子序列长度
    int i=m,j=n;
    while (k>0) {
        if (dp[i][j]==dp[i-1][j])
            i--;
        else if(dp[i][j]==dp[i][j-1])
            j--;
        else {
            subs+=a[i-1];      //subs中添加a[i-1]
            i--; j--; k--;
        }
    }
    reverse(subs.begin(),subs.end());
    return subs;              //返回逆置subs的字符串
}

```


算法空间优化*

将 dp 改为一维数组，在阶段 $i-1$ （指考虑 $a[i-1]$ 字符的阶段）将 $dp[i-1][j-1]$ 存放在 $dp[j-1]$ 中，将 $dp[i-1][j]$ 存放在 $dp[j]$ 中，这两个状态是可以区分的。

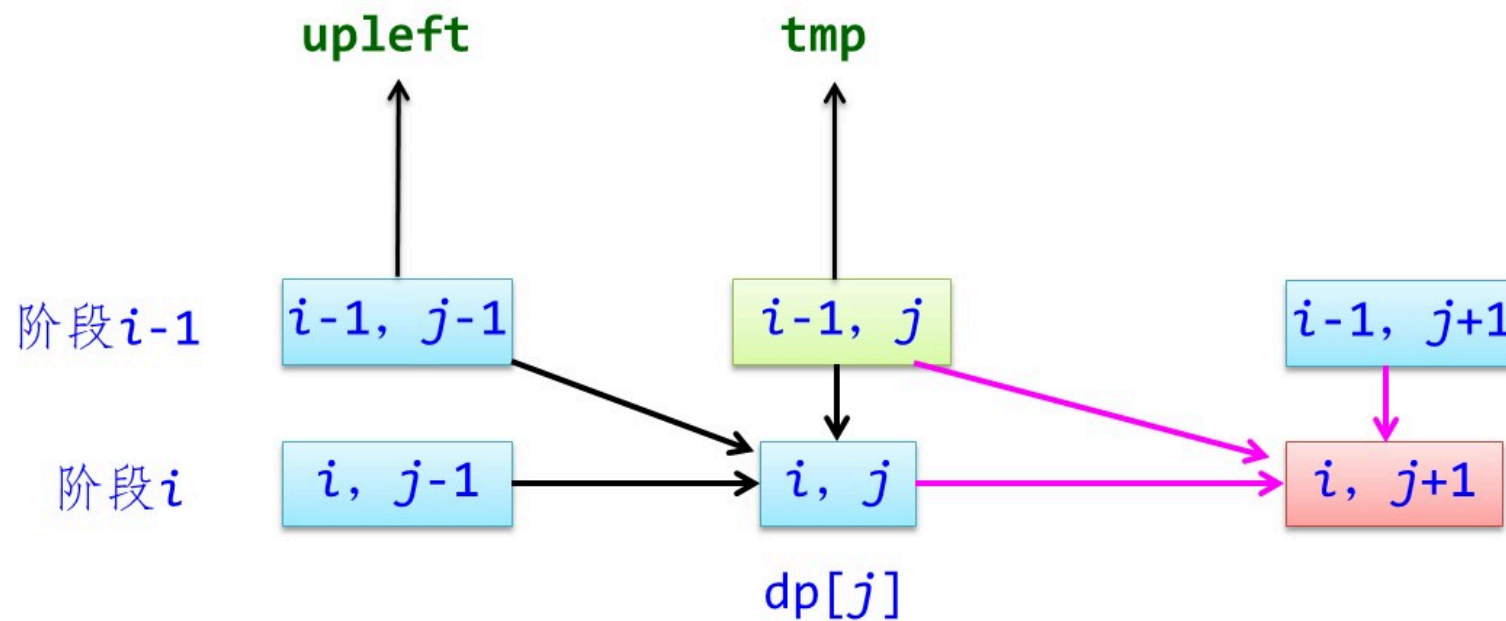


$$dp[i][j] = dp[i-1][j-1] + 1$$

$$a[i-1] = b[j-1]$$

$$dp[i][j] = \max\{dp[i][j-1], dp[i-1][j]\}$$

$$a[i-1] \neq b[j-1]$$



```

int LCSlength1(string&a,string&b) { //求LCS的改进算法
    int m=a.size(); //m为a的长度
    int n=b.size(); //n为b的长度
    int dp[n+1]; //一维动态规划数组
    memset(dp,0,sizeof(dp));
    for (int i=1;i<=m;i++) {
        int upleft=dp[0]; //阶段i初始化upleft
        for (int j=1;j<=n;j++) {
            int tmp=dp[j]; //临时保存dp[j]
            if (a[i-1]==b[j-1])
                dp[j]=upleft+1; //修改dp[j]
            else
                dp[j]=max(dp[j-1],dp[j]);
            upleft=tmp; //更新upleft为dp[j]修改之前的值
        }
    }
    return dp[n];
}

```



青岛科技大学
Qingdao University of Science & Technology

数据结构与算法分析2

第7章 动态规划1—基本原理

梁宏涛

lht@qust.edu.cn

周二&周四上午

38

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

7.6 编辑距离

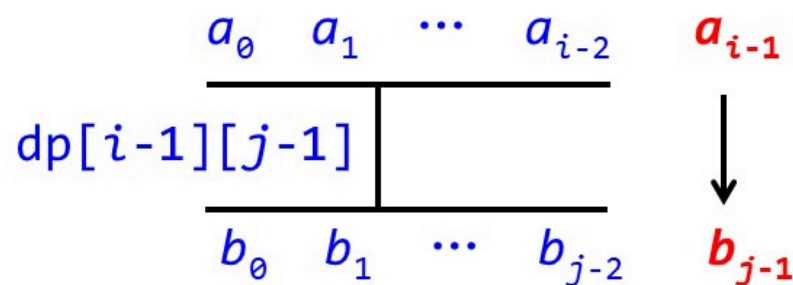
- 设 a 和 b 是两个字符串。现在要用最少的字符操作次数（最优编辑距离），将字符串 a 编辑为字符串 b 。这里的字符编辑操作共有3种：删除一个字符，插入一个字符或者将一个字符替换另一个字符。
- 例如， $a="sfdqxbw"$ ， $b="gfdgw"$ ，结果为4。

$a \Rightarrow b$

解

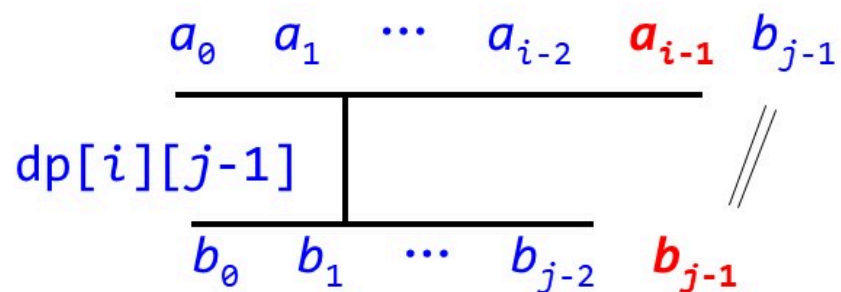
- 设计二维动态规划数组 dp ，其中 $dp[i][j]$ 表示将 $a[0..i-1]$ ($1 \leq i \leq m$) 编辑为 $b[0..j-1]$ ($1 \leq j \leq n$) 的最优编辑距离（即最少编辑操作次数）。
- 当 b 为空串时，要删除 a 中全部字符得到为 b ，即 $dp[i][0]=i$ （删除 a 中 i 个字符，共 i 次操作）。
- 当 a 为空串时，要在 a 中插入 b 的全部字符得到 b ，即 $dp[0][j]=j$ （向 a 中插入 b 的 j 个字符，共 j 次操作）。

- 当两个字符串 a 、 b 均不空时，若 $a[i-1]=b[j-1]$ 时，这两个字符不需要任何操作，即 $dp[i][j]=dp[i-1][j-1]$ 。
- 当 $a[i-1] \neq b[j-1]$ 时，以下3种操作都可以达到目的：
 - ① 将 $a[i-1]$ 替换为 $b[j-1]$ ， $dp[i][j]=dp[i-1][j-1]+1$ （一次替换操作的次数计为1）。



(a) 替换

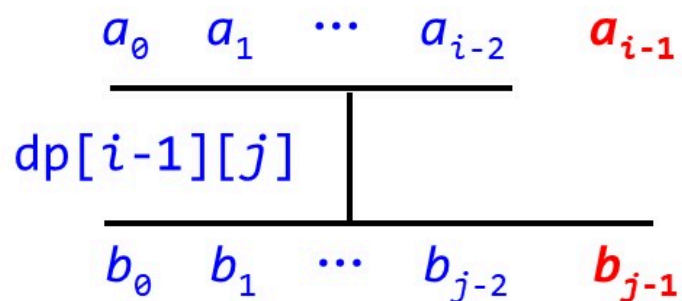
② 在 $a[i-1]$ 字符后面插入 $b[j-1]$ 字符，有
 $dp[i][j]=dp[i][j-1]+1$ （一次插入操作的次数计为1）。



(b) 插入

③ 删除 $a[i-1]$ 字符, 有 $dp[i][j]=dp[i-1][j]+1$ (一次删除操作的次数计为1)。

■ 此时 $dp[i][j]$ 取上述三种操作的最小值。



(c) 删除

状态转移方程

$dp[i][0]=i$

边界情况

$dp[0][j]=j$

边界情况

$dp[i][j]=dp[i-1][j-1]$

当 $a[i-1]=b[j-1]$ 时

$dp[i][j]=\min(dp[i-1][j-1]+1, dp[i][j-1]+1, dp[i-1][j]+1)$

当 $a[i-1]\neq b[j-1]$ 时

最后得到的 $dp[m][n]$ 即为所求。

```

int editdist(string&a,string&b) {    //求a到b的编辑距离
    int m=a.size();
    int n=b.size();
    int dp[m+1][n+1];                //二维动态规划数组
    memset(dp,0,sizeof(dp));
    for (int i=1;i<=m;i++)
        dp[i][0]=i;                  //把a的i个字符全部删除转换为b
    for (int j=1; j<=n;j++)
        dp[0][j]=j;                  //在a中插入b的全部字符转换为b
    for(int i=1;i<=m;i++) {
        for(int j=1;j<=n;j++) {
            if (a[i-1]==b[j-1])
                dp[i][j]=dp[i-1][j-1];
            else
                dp[i][j]=min(min(dp[i-1][j-1],
                                dp[i][j-1]),dp[i-1][j])+1;
        }
    }
    return dp[m][n];
}

```

 **editdist**算法分析：该算法中包含两重**for**循环，对应的时间复杂度为 $O(m \times n)$ ，空间复杂度为 $O(m \times n)$ 。

提纲
CONTENTS

第7章—动态规划

7.1 动态规划概述

7.2 最大连续子序列和

7.8*完全背包&多重背包问题

7.3 最长递增子序列

7.9扔鸡蛋问题

7.4 三角形最小路径

7.10资源分配问题

7.5 最长公共子序列

7.11旅行商问题

7.6 编辑距离

7.12最少士兵数问题

7.7 0/1背包问题

7.13矩阵连乘问题

7.7 0/1背包问题

 **问题描述：**有 n 个物品，物品编号为 $0 \sim n-1$ ，重量为 $\{w_0, w_1, \dots, w_{n-1}\}$ ，价值为 $\{v_0, v_1, \dots, v_{n-1}\}$ ，给定一个容量为 W 的背包。现在从这些物品中选取若干物品装入该背包的方案，每个物品要么选中要么不选中，要求选中的物品总重量不超过背包容量并且具有最大的价值。

解

- 设置二维动态规划数组 **dp**。
- **dp[i][r]** 表示在物品 $0 \sim i-1$ （共 i 个物品）中选择物品并且背包容量为 r （ $0 \leq r \leq W$ ）时最大价值，或者说只考虑前 i 个物品并且背包容量为 r 时最大价值。

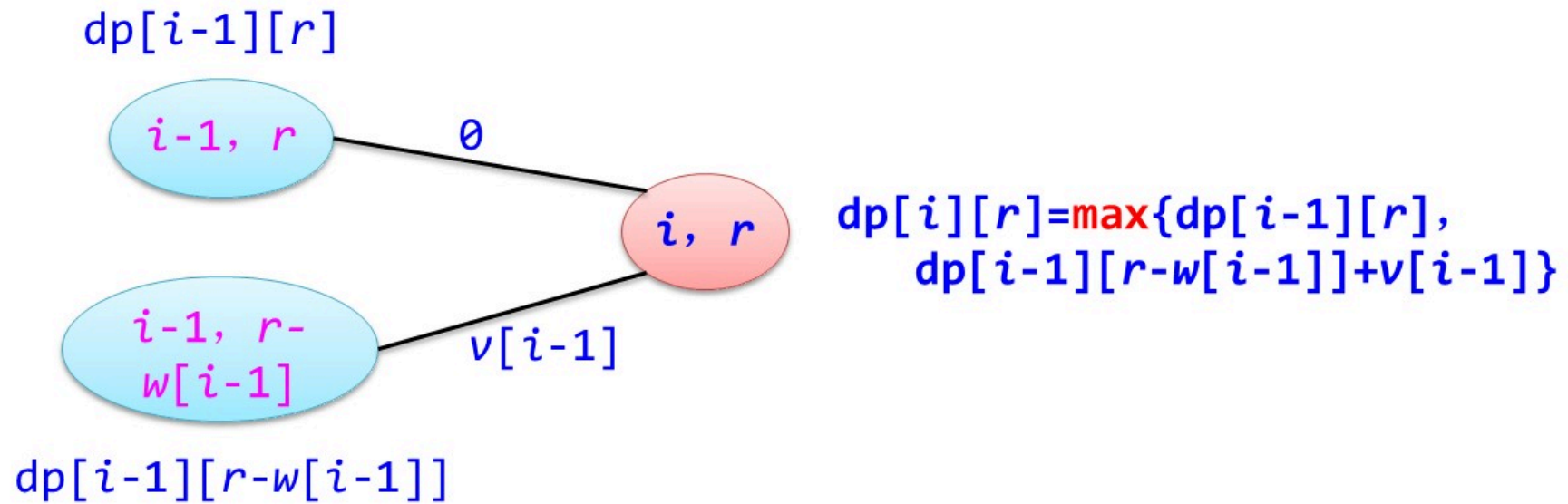
当考虑物品 $0 \sim i-2$ 之后，已经求出相应的 dp 元素，现在考虑物品 $i-1$ ，分为两种情况：

1. 若 $r < w[i-1]$ ，说明物品 $i-1$ 放不下，此时等效于只考虑前 $i-1$ 个物品并且背包容量为 r 时的最大价值，所以有 $dp[i][r] = dp[i-1][r]$ 。

2. 若 $r \geq w[i-1]$, 说明物品 $i-1$ 能够放入背包, 又有两种子情况:

- 不选择物品 $i-1$ 即不将物品 $i-1$ 放入背包, 等同于情况1。
- 选择物品 $i-1$ 即将物品 $i-1$ 放入背包, 这样消耗了 $w[i-1]$ 的背包容量, 获取了 $v[i-1]$ 的价值, 那么留给前 $i-1$ 个物品的背包容量就只有 $r-w[i-1]$ 了, 此时的最大价值为 $dp[i-1][r-w[i-1]]+v[i-1]$ 。

状态转移图



状态转移方程

$dp[i][0]=0$ (没有装入任何物品, 总价值为0)

边界情况

$dp[0][r]=0$ (没有考虑任何物品, 总价值为0)

边界情况

$dp[i][r]=dp[i-1][r]$

当 $r < w[i-1]$ 时, 物品 $i-1$ 放不下

$dp[i][r] = \max\{dp[i-1][r], dp[i-1][r-w[i-1]]+v[i-1]\}$

否则在不放入和放入物品 $i-1$ 之间取最大价值

在求出 dp 数组后 $dp[n][W]$ 元素就是答案。

```

vector<vector<int>> dp;                                //二维动态规划数组
int knap(int w[],int v[],int n,int W) {
    dp=vector<vector<int>>(n+1,vector<int>(W+1));
    for (int i=0;i<=n;i++) dp[i][0]=0;    //置边界条件
    for (int r=0;r<=W;r++) dp[0][r]=0;    //置边界条件
    for (int i=1;i<=n;i++) {
        for (int r=0;r<=W;r++) {
            if (r<w[i-1])
                dp[i][r]=dp[i-1][r];
            else
                dp[i][r]=max(dp[i-1][r],dp[i-1][r-w[i-1]]
                    +v[i-1]);
        }
    }
    return dp[n][W];
}

```

 **knap算法分析**：该算法中包含两重**for**循环，所以时间复杂度为 $O(nW)$ ，空间复杂度为 $O(nW)$ 。

★ 表面上看其时间函数是 n 的多项式，但实际上并不是真正多项式级的算法。如果 W 不是太大，则 nW 要远好于一个 n 的指数级。如果 W 非常大，或者为很大的实数，则该动态规划方法不可行，所以说这样的算法是伪多项式时间的算法。

当求出dp数组后，如何推导出一个解向量 $x = (x_0, x_1, \dots, x_{n-1})$ 呢？从前面状态转移方程的后两行看出：

- 若 $dp[i][r] = dp[i-1][r]$ ，表示物品 $i-1$ 放不下或者不放入物品 $i-1$ 的情况，总之不选择物品 $i-1$ 。也就是说当前dp元素等于上方元素，不选择对应的物品（物品 $i-1$ ），并跳到上方位置。
- 否则一定有 $dp[i][r] \neq dp[i-1][r]$ 成立，表示选择物品 $i-1$ 。也就是说当前dp元素不等于上方元素，选择对应的物品，并跳到左上角 $dp[i-1][r-w[i-1]]$ 的位置。

$dp[i][r] = dp[i-1][r]$

当 $r < w[i-1]$ 时，物品 $i-1$ 放不下

$dp[i][r] = \max\{dp[i-1][r], dp[i-1][r-w[i-1]] + v[i-1]\}$

否则在不放入和放入物品 $i-1$ 之间取最大价值

程序
验证

$n=5$, $w=\{2, 2, 6, 5, 4\}$, $v=\{6, 3, 5, 4, 6\}$, $W=10$ 。



dp:

0	0	0	0	0	0	0	0	0	0	0
0	0	6	6	6	6	6	6	6	6	6
0	0	6	6	9	9	9	9	9	9	9
0	0	6	6	9	9	9	9	11	11	14
0	0	6	6	9	9	9	10	11	13	14
0	0	6	6	9	9	12	12	15	15	15

求解结果<最优方案>
选取的物品: 0 1 4
总价值=15

$i \backslash r$	0	1	2	3	4	5	6	7	8	9	10
$w_0=2$	0	0	0	0	0	0	0	0	0	0	0
$w_1=2$	1	0	0	6	6	6	6	6	6	6	6
$w_2=6$	2	0	0	6	6	9	9	9	9	9	9
$w_3=5$	3	0	0	6	6	9	9	9	11	11	14
$w_4=4$	4	0	0	6	6	9	9	9	10	11	13
5	0	0	6	6	9	9	12	12	15	15	15

选择物品4
选择物品1
选择物品0

- 若 $dp[i][r] = dp[i-1][r]$ ，当前 dp 元素等于上方元素，不选择物品 $i-1$ ，并跳到上方位置，即 $dp[i][r] \Leftarrow dp[i-1][r]$ 。
- 否则当前 dp 元素不等于上方元素，选择对应的物品 $i-1$ ，并跳到左上角 $dp[i-1][r-w[i-1]]$ 的位置，即 $dp[i][r] \Leftarrow dp[i-1][r-w[i-1]]$ 。

```

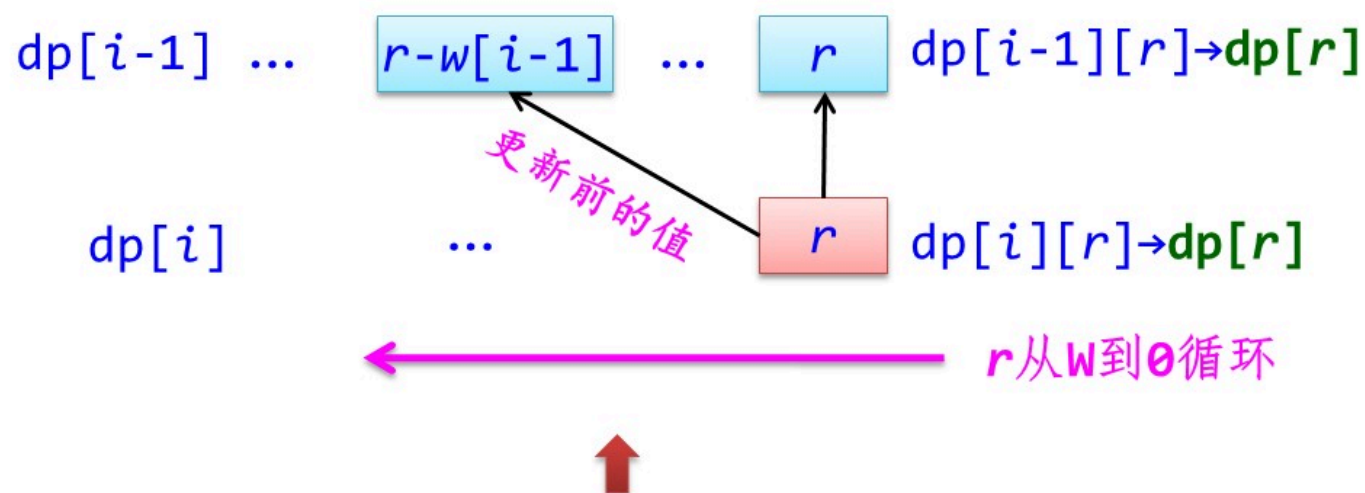
void getx(int w[],int n,int W) { //回推求一个最优方案
    vector<int> x(n,0);          //解向量
    int i=n,r=W;
    while (i>=1) {
        if (dp[i][r]!=dp[i-1][r]) {
            x[i-1]=1;           //选取物品i-1
            r=r-w[i-1];
        }
        else x[i-1]=0;          //不选取物品i-1
        i--;
    }
    printf("    选择的物品:");
    for(int i=0;i<n;i++) {
        if(x[i]==1) printf(" 物品%d",i);
    }
    printf("\n");
}

```

算法空间优化*

- $dp[MAXN][MAXW]$ 改为一维数组 $dp[MAXW]$ 。

$dp[i-1][r-w[i-1]] \rightarrow dp[r-w[i-1]]$



$dp[i][r] = dp[i-1][r]$
 $dp[i][r] = \max\{dp[i-1][r], dp[i-1][r-w[i-1]] + v[i-1]\}$

```

int knap1(int w[],int v[],int n,int W) {    //改进算法
    vector<int> dp(W+1,0);                //一维动态规划数组
    for (int i=1;i<=n;i++) {
        for (int r=W;r>=0;r--) { ?        //r按0到W的逆序（重点）
            if (r<w[i-1])
                dp[r]=dp[r];
            else
                dp[r]=max(dp[r],dp[r-w[i-1]]+v[i-1]);
        }
    }
    return dp[W];
}

```

可以等价地改为如下

```
int knap2(int w[],int v[],int n,int W) {    //改进算法
    vector<int> dp(W+1,0);                //一维动态规划数组
    for (int i=1;i<=n;i++) {
        for (int r=W;r>=w[i-1];r--) //r按w[i-1]到W的逆序（重点）
            dp[r]=max(dp[r],dp[r-w[i-1]]+v[i-1]);
    }
    return dp[W];
}
```